Reviewing options for fine-grained Role-Based Access Control in Data Lakes

Andreas Severin Hauch Trøstrup
Computer Science
IT University of Copenhagen
atro@itu.dk

Lucas Frey Torres Hanson
Computer Science
IT University of Copenhagen
luha@itu.dk

***Abstract*—Data lakes today have limitations in only allowing fine-grained access control on fully-managed cloud platforms like Snowflake or Databricks, where open data lakes with potentially many different query engines have to rely on limited catalogue-based access control.**

Theory and proposed solutions will be examined to see which options might be able to bridge this gap of ensuring fine-grained access control, and if encryption and role-based access control might be part of solving it.

We propose a system for enforcing fine-grained role-based access control at the object store level as well as creating a wrapper around the object store to allow query engines to access the data without modifications.

Our findings show that the encryption to enforce access control might be a viable solution, as both modified and unmodified query engines are enforced to follow the column-level granularity using the role-based access control if they want to read the data, but will also have to be proven functional by a future MVP.

I. Introduction

Today, only a few data lake solutions implement Role-Based Access Control (RBAC) down to individual files, singular columns, and specific entries. When implemented, access control is often delegated to external services or is part of a proprietary solution. Databricks[1] enforces row and column level permissions on a version of Parquet, but is partly a proprietary solution, and Snowflake[2] has its own proprietary solutions as well. [1], [2] This means that most engines, which do not themselves handle granularity, will have access to whole files at a time, instead of, e.g. a single column of the given file. Recent efforts like the Lakekeeper[3] project or Apache Polaris[4] attempt to address this gap by creating open-source solutions for secure access, but these come with limitations in their ability for fine-grained access control (FGAC), and are still in ongoing development.

This creates an issue for data lakes requiring access control, for example, if they contain financial, personal or GDPR-protected data, as owners of the data must either choose to commit to a single well-defined data lake system, thus ensuring FGAC, or if they also want third-party query engines to access the data, the granularity will now only be on the file level. Solving this, data lakes could be a tool for more varied applications, as multiple different solutions can share data while still keeping their sensitive information secret.

By reading about current, proposed, and researched solutions, their individual advantages and disadvantages can be identified. We will review available information on proprietary approaches, but focus on open-source and open-standard proposals to see how authorization might be ensured for everyone with object store access – such as with encryption. This will be used for a literature survey on fine-grained RBAC in data lakes, examining at least two open-source systems, including Lakekeeper [3] and Apache Iceberg [4], and comparing their respective feature sets to proprietary solutions. Afterwards, we will propose a solution which can unify RBAC in data lake solutions, and based on the findings, we will examine the viability of this solution. This will be ground groundwork for a later minimal viable product (MVP) for a thesis project.

This paper is meant to lay the groundwork and create guidelines as to how to create a data lake system, which allows various query engines to access the data, whilst ensuring that access control is enforced, so trust in how the query engine handles the granularity of said access control is not needed.

II. Background

Though the concept of data lakes has existed since the 2000s, the term was only coined in 2011 by James Dixon[5]. Most of the current solutions are thus still in active development, and often in a pre-release. This means that the documentation used might have been changed multiple times during writing.

In this section, we will discuss multiple approaches and technologies used within data lake systems.

A. Role-based Access Control

Role-based Access Control (RBAC) [6] is an approach to access control based around assigning permissions to roles, and grouping users into these roles. Access control is then managed by controlling what roles are allowed to do what, instead of managing each user individually. The "Core RBAC" model, as defined by Ferraiolo et. al. 2001, [7], consists of five basic elements: *users*, *roles*, *permissions*, *operations* and *objects*. A *user* is some actor, typically a human user, but could also be an autonomous actor, who needs to be authorized for some set of actions. A user is assigned one or more *roles*. A role is simply some named collection of authorizations; for example, a job title. A role is assigned

[1]Section IV.C defines Databricks
[2]Section IV.B defines Snowflake
[3]Section IV.A defines Lakekeeper
[4]Section IV.D defines Apache Polaris

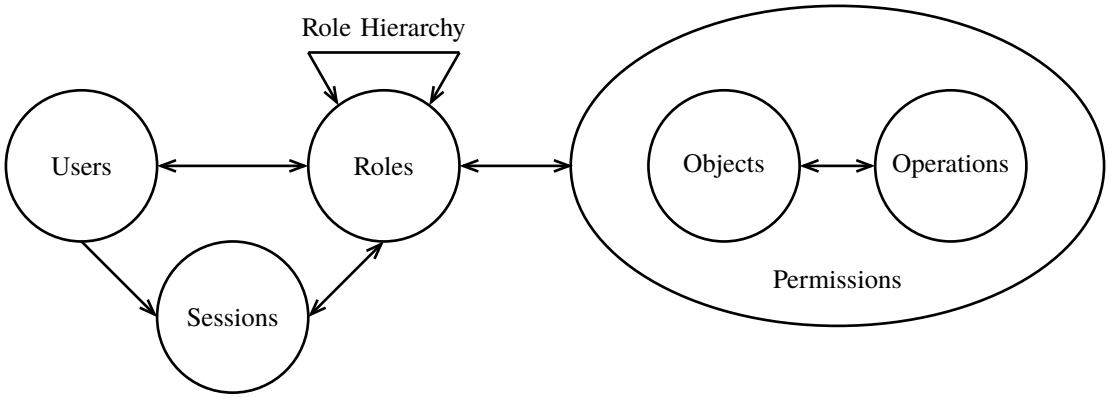

Fig. 2. The entity relationships in Hierarchical RBAC, as described by Ferraiolo et. al. 2001 [7]. Arrows represent a many-to-many relation.

```
GRANT SELECT, INSERT ON TABLE journals TO doctors;
```

Operations: SELECT, INSERT; Object: TABLE journals; Permission: SELECT, INSERT ON TABLE journals; Role: doctors

```
GRANT doctors TO alice;
```

Role: doctors; User: alice

Fig. 1. Assignment of permission to a role, and role to a user in SQL

one or more *permissions*, which are what give authorization to perform an *operation* on a secured *object*. In a database context, an operation could be SQL commands like `SELECT` or `INSERT`, which are then applied to some database object, like a table or column. With this, *users* have a many-to-many relationship to what *permissions* they are authorized for, through their roles. Figure 1 shows an example of permissions assignment to a role, and role assignment to a user in SQL.

In addition, Core RBAC also describes *sessions*. A *session* is a mapping of one user to a subset of the roles they are authorized for; i.e. a user "activates" one or more roles that have the necessary permissions, when they need to perform a specific operation on an object. This allows for finer control of roles that are active in a given context, to prevent unnecessary access levels. Commonly, a hierarchy is introduced to allow for structuring roles which should inherit some base permissions; for example, a Doctor and Nurse role might inherit permissions from some base "Healthcare Worker" role. This is referred to as "Hierarchical RBAC" [7].

User membership is inherited top-down, and role permissions are inherited bottom-up. That is, a "Doctor" is also a member of the "Healthcare Worker" role, and shares all the permissions of the role, including some of its own. Figure 2 shows the relationship of the entities in Hierarchical-, and by extension, Core RBAC.

B. Encryption-at-Rest

Encryption-at-rest is another fundamental part of data security in databases. While access control methods like RBAC govern access to the data, encryption-at-rest protects the data itself by encrypting the data as it is stored on storage devices. This ensures that even if leaks occur, due to, for example, misconfigured permissions in RBAC, attackers would also need to decrypt the data to access any information. Encryption is typically performed using a strong symmetric key encryption algorithm like AES-256. [8]

C. Data Lakes

A "data lake" is a centralized repository of data, which could be structured, semi-structured or unstructured.[5] It differs from a traditional database in that a data lake separates compute and storage layers. The data itself is stored in object stores like Amazon S3[9], in file formats like CSV, Parquet[5], etc. The compute nodes are simply any query engine, such as Apache Spark [10], that connects to the object store and uses the data. This could be a data scientist working on their own machine, who connects to a data lake and runs queries; or it could be a part of a managed all-in-one platform like Snowflake[6]. As the object store just contains raw data, both structured and unstructured, this can make it more complicated to query, especially for large data lakes. One approach to querying is "schema on read", in which the query engine infers a schema at query time based on the files in the data lake. This is in contrast to "schema-on-write" used in traditional relational databases, where the schema is strictly enforced when writing. This doesn't work well for data lakes due to the nature of storing raw data in object stores. Typically, data lakes introduce a *catalogue* on top of the object store that manages file schemas, metadata and snapshots, allowing for structured reads and more performant querying through techniques like partitioning, i.e. filtering unnecessary data. This has also been referred to as a *data lakehouse* .[11]

D. Identity Provider

Identity Provider (IdP) is responsible for storing and managing a user's identity, thus ensuring authentication. When provided with a valid password and email, it will return a token, most often a JWT Token[7], proving the client's identity. This token is then appended to subsequent calls, which need the user to be authenticated.[12] The IdP Service is one of the external services some data lakes rely on for validating identities, while some have their own internal one, or sometimes they have the option for both.

[5]Section II.H defines Parquet

[6]Section IV.B defines Snowflake

[7]https://www.ibm.com/docs/en/cics-ts/6.x?topic=cics-json-web-token-jwt

E. Fine-grained Access Control

Access control (AC) is the ability to restrict access to data, while fine-grained access control (FGAC) is the ability to restrict access to some degree finer than a whole table. This can vary in granularity, but IBM mentions it as a definition that covers row and column level access control.[13] Hence, if a client tries to access or modify some data, then based on some policy, they either have access to the given data or do not. This is useful for when you want to have a client only be able to perform some sub-scoped CRUD actions on tables, whilst another client has different CRUD permissions.

In this paper, the FGAC will mostly be used to reference granularity on the column level, as this is the most commonly supported granularity, and is also supported by Parquet with encryption, discussed in Section II.H. If row granularity is also ensured by some system, then it will be explicitly written, and not referred to as only FGAC.

F. Key Management System

A key management system (KMS) is a system that securely stores cryptographic keys and the metadata associated with them. Along with storing the cryptographic keys, it sometimes also handles, for example, distribution and recovery of the keys.[14] The KMS is often provided as a service, e.g., AWS KMS, where KMS here stands for Key Management Service.[15]

The reason for having a KMS is most often to store symmetric cryptographic keys, but it can also be for asymmetric ones, so data is safe from attacks even if leaked from the store, and also so the keys can be distributed to authorized users. So its major purpose is to enforce key policies.[16]

We will use KMS to refer to "key management system", thus relying on the more lenient definition, and will specify if referring to the KMS service.

G. Iceberg

Iceberg [17] is an open-source "table format" for large analytical tables. It provides query engines with a standardized way of understanding tables in data lakes and enables performant and reliable querying. Iceberg organizes data in an object store into tables with formal schemas, provides partitioning using file metadata and provides version snapshots of these tables. This allows query engines to query the data lake in a structured, reliable way with standard SQL, and allows for fast reads even in petabyte-scale tables, due to partitioning using the metadata. It also allows for *time travel* queries with help from the versioned snapshots, which allows analysts to make reproducible queries as they were at some point in time.

While Iceberg is not in itself a catalogue, it provides a REST API Specification which can be used to implement an Iceberg-based catalogue, like Lakekeeper (Section IV.A) or Polaris (Section IV.D).

H. Parquet

Parquet [18] is a columnar file format supported by many data processing systems.

A Parquet file is structured using row groups, which contain columns, along with a footer which contains metadata. The columns are structured as column chunks, which contain several *data pages*. A data page contains a header, which describes information like the size of the page, and following that, the actual values as bytes. The file footer contains metadata that specifies what is stored in the file and where, that is, where a reader should look to find a specific column, or the statistics of a column (e.g. min/max). Figure 3 shows a visual representation of a Parquet file that also uses encryption, which will be described later. The left side of the figure shows row groups, columns and data pages, along with their headers. The right side shows the footer of the file, with the file metadata containing metadata on each row group, which in turn contains metadata on the column chunks in that row group. The arrows link information about where to read, such as offsets, to where they point the reader. The objects that hold metadata, like statistics, and structural information, including physical offsets, such as the footer and page headers, are serialized using Apache Thrift [19], a language-agnostic serialization framework, and referred to as "Thrift Structs". Essentially, the object that contains the information, for example, a `PageHeader` object (on the left side of Figure 3) or a `ColumnMetaData` object (on the right side of Figure 3), is serialized into bytes using Thrift, which can then later be restored to the same object.[20] Then, to read a Parquet file, the reader de-serializes the Thrift structures to get the necessary metadata, and then reads the actual data values using the information from the metadata.

a) *Encryption in Parquet:*

Parquet Modular Encryption was introduced to allow granular control of how the Parquet file data and metadata should be encrypted. It allows the user to either encrypt whole files using a single key, encrypt singular columns using one or more keys, and choose whether or not the file footer should also be encrypted. If the file footer could reveal sensitive information, e.g. min or max values, encrypting it is likely a good idea.

Internally, using encryption means that the serialized Thrift structs are encrypted using the specified key, and the data pages themselves are also encrypted. If footer encryption is enabled, a new struct is added to the file: `FileCryptoMetaData`, which specifies the encryption key ID of the footer (for retrieval by a KMS) and the algorithm used. This struct can be seen in Figure 3, at the top of the right side, i.e. the footer. The key specified by this struct can be used to decrypt the footer, which in turn stores information about which columns are encrypted using what keys, using the `ColumnCryptoMetaData` struct stored for each row group. The information in this struct can be used to decrypt the column metadata and the column data itself. Figure 3 also shows what parts of the file is encrypted with that key; the red key shows what is encrypted using the column key, and the black key what is encrypted using the footer key.

III. Related Work

This section will describe existing research done on access

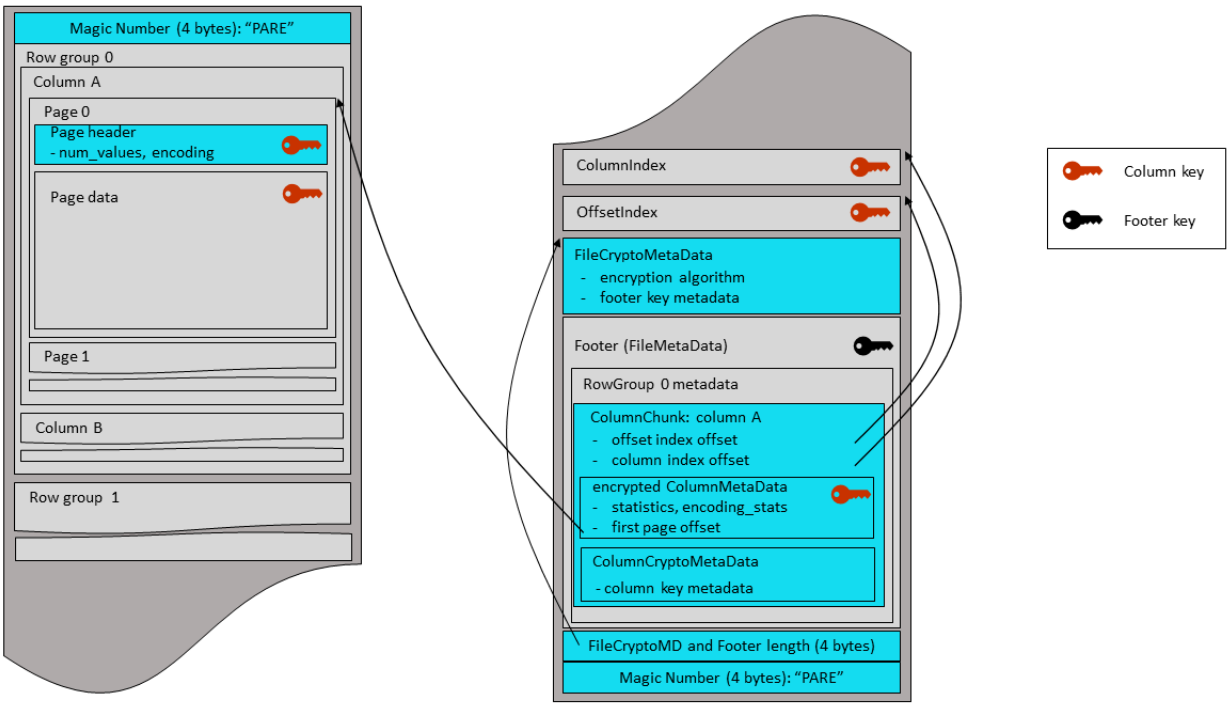

Fig. 3. Illustration of the Parquet file structure using an Encrypted Footer. From [20].

control in data lakes and how it can use encryption-at-rest[8]. This has direct relevance to the proposal made later in the paper.

A. *"One Stone, Three Birds: Finer-Grained Encryption with Apache Parquet @ Large Scale"*

Shang et.al. 2022 [21] proposed a system for Uber Technologies, which is very similar to the "FGRBAC by Encryption"[9] solution. In the paper, they also find that the modern "data lakehouse"[10] has an issue in terms of the underlying data not necessarily being secured, and also describe access being governed at the engine or catalogue level as problematic.

Similarly, they describe using Parquet[11] file encryption to achieve column-level access control by creating column-specific encryption keys and storing these in a Key Management Service[12]. In their system, the writers tag the datasets at the column level if encryption is needed, and when writing the Parquet data, they encrypt the tagged columns using the specified keys. The encrypted data is then written to the data lake[13], and on read, the readers decrypt the data if they have access to the necessary keys. This solves encryption-at-rest and access control just like our proposed solution. However, a key difference is that their system relies on an extension of the Parquet library to handle encryption and decryption, as the cryptographic operations are performed by the readers and writers. They propose adding a plugin to handle this, but the responsibility would still be at the query engine level, even for readers. This is in contrast to the proposed solution, where readers will be able to fetch the data through the Object Store Wrapper Service[14] that automatically decrypts data if the necessary keys are accessible.

B. *Membrane: A Cryptographic Access Control System for Data Lakes*

Membrane is a proposal for a data lake system where access control is enforced by encryption, written by Kumer et.al. 2025 [22]. The idea is to encrypt all data before storing it in the object stores, thus achieving encryption-at-rest. It further takes advantage of that the compute resources are both physically and logically distinct. Thus, if the attacker gains access to the object store, they will not have access to the virtual machine itself, meaning that the symmetrically encrypted data is kept safe.

They propose encrypting all cells individually, thus achieving a much finer-grained access control, compared to Parquet's own supported column-level granularity. This is achieved by that if a data owner wants to insert data into S3, then they first generate a cryptographic key for each table. The table key, together with row and column ID, infers a cell cryptographic key. Then each cell can be encrypted, and the whole table can be inserted into S3. To then allow read access to others, a cryptographic view is created, allowing for decrypting only the data intended by the data owner. But this results in a

[8]Section II.B defines encryption-at-rest
[9]Section V
[10]Section II.C defines lakehouse
[11]Section II.H defines Parquet
[12]Section II.F also defines Key Management Service
[13]Section II.C defines data lakes
[14]Section V.-.b defines Object Store Wrapper Service (OSWS)

query where each cell needs to be decrypted individually, to get usable data from them.[22]

IV. State of Access Control in Data Lakes

In this section, we will describe how some available data lake solutions handle AC, how fine-grained they are and how they manage authorization. Most of them use either Parquet or Iceberg to structure the data to some extent in the object store, mentioned in Section II.

A. Lakekeeper

Lakekeeper [3] is an open-source implementation of the Apache Iceberg REST catalogue API. Like with other Iceberg-based catalogues, see Section II.G, it acts as the metadata store for the data lake, with the data itself residing in a remote object store like Amazon S3. Lakekeeper adds a security layer on top of the data lake by adding dependencies on several external services, which it uses for authentication and authorization. While Lakekeeper *can* be used without authentication enabled, the following assumes it is enabled.

Persistence backend Lakekeeper requires a persistence backend for the catalogue metadata.

It currently only supports a PostgreSQL database.

Object store An object store is required for a Lakekeeper-based data lake, as this is where the data resides.

A key difference is that access to the object store is governed only by Lakekeeper; i.e. no one other than Lakekeeper itself can have "direct" access to the object store without permission by Lakekeeper.

Identity provider Lakekeeper requires an external Identity Provider to authenticate users.

The recommended approach is to use any identity provider supporting OpenID Connect (OIDC)[15].

Authorization System Lakekeeper requires an authorization system to manage permissions and perform authorization checks. An example is OpenFGA [24].

OpenFGA is an open-source *relationship-based* fine-grained authorization system based on Google Zanzibar, Google's internal authorization system, first described by Pang. et. al [25].

a) *OpenFGA:* OpenFGA provides a platform for modelling authorization using an "authorization model", a schema that defines existing types in the system, like objects or users. To define permissions, *relationship tuples* are added to the system, which define a relationship between a user and an entity – the collection of relationships then acts as the permissions in the system. Later, to check permissions, a "check request" can be sent to the API. This request asks: "Does *some user* have *some relationship* with *some entity*?".

The following describes the flow of authentication and authorization in Lakekeeper. When a query engine then connects to Lakekeeper, it uses the above components to evaluate whether or not the request is permitted. Figure 4 shows a sequence diagram of this flow, using Vended Credentials. To perform a query, the client/query engine obtains a JWT token from some OIDC-capable identity provider, like Microsoft Entra ID[16] or Keycloak.[27], steps 1 to 2 in Figure 4. The query engine then provides the acquired token together with its call to Lakekeeper, step 3. Lakekeeper proceeds to verify the token against the configured IdP, steps 4 and 5.

Given Lakekeeper was provided a valid token, the user is added to its internal database if the user did not already exist. Lakekeeper now contacts OpenFGA, the default permission system, to check if the user has the required relation to the requested object, to authorize the request, steps 6 and 7. If the request is authorized, it will return an access mode to the client. This will enable them to do the required operation on the store, and they get access to the store through an access mode, steps 8 to 10. For Amazon S3, Lakekeeper supports *remote request signing* or *vended credentials* [28].

Remote Signing The query engine sends an unsigned or self-signed S3 request to Lakekeeper, which then checks if the request is permitted. If so, Lakekeeper signs it with its secure credentials and returns it to the client, who can then use it to access the permitted files in S3 directly.

Vended credentials Vended credentials are a way to give an entity access to a location using temporary credentials. Lakekeeper uses Amazon Secure Token Service (STS) [29], to create temporary, fine-grained credentials that the query engine can use to access S3.

Lakekeeper offers a hierarchical granularity system, starting at the *server* level; a Server refers to a single, or a cluster of, Lakekeeper pod(s), that is, individual running instances of the application, with common state. A *server* can contain one or more *projects*, which in turn contain *warehouses*. A warehouse refers to the data lake itself; it is the entity that query engines connect to. A warehouse contains *namespaces*, which contain *tables*, *views* or other namespaces. Access to each of these levels can be controlled using the permission system. Tables and views are thus the "finest" level of granularity supported by Lakekeeper. Figure 5 shows the hierarchy.

Using OpenFGA, Lakekeeper supports data access control, but it offers no mechanism for encryption-at-rest. However, due to Lakekeeper being an Iceberg catalogue and thus supporting Parquet files for storage, encryption-at-rest can be achieved using encryption of the stored Parquet files (Section II.H.a). This would, however, require a manual intervention or a separate system to manage cryptographic operations on the data at rest and the handling of the cryptographic keys.

B. Snowflake

The "Snowflake Elastic Data Warehouse", or Snowflake for short, [30] is a fully cloud-based Software-as-a-Service (SaaS) solution for data analysis.

It is built upon three architectural layers of independent services: the *data storage* layer, which handles data storage in an object store such as S3, the *virtual warehouse* layer, which handles query execution through an elastic cluster of worker nodes, and the *cloud services* layer, a collection

[15] OIDC is an identity protocol which extends OAuth 2.0[23]

[16] Microsoft Entra ID being an Identity and Access Management System[26]

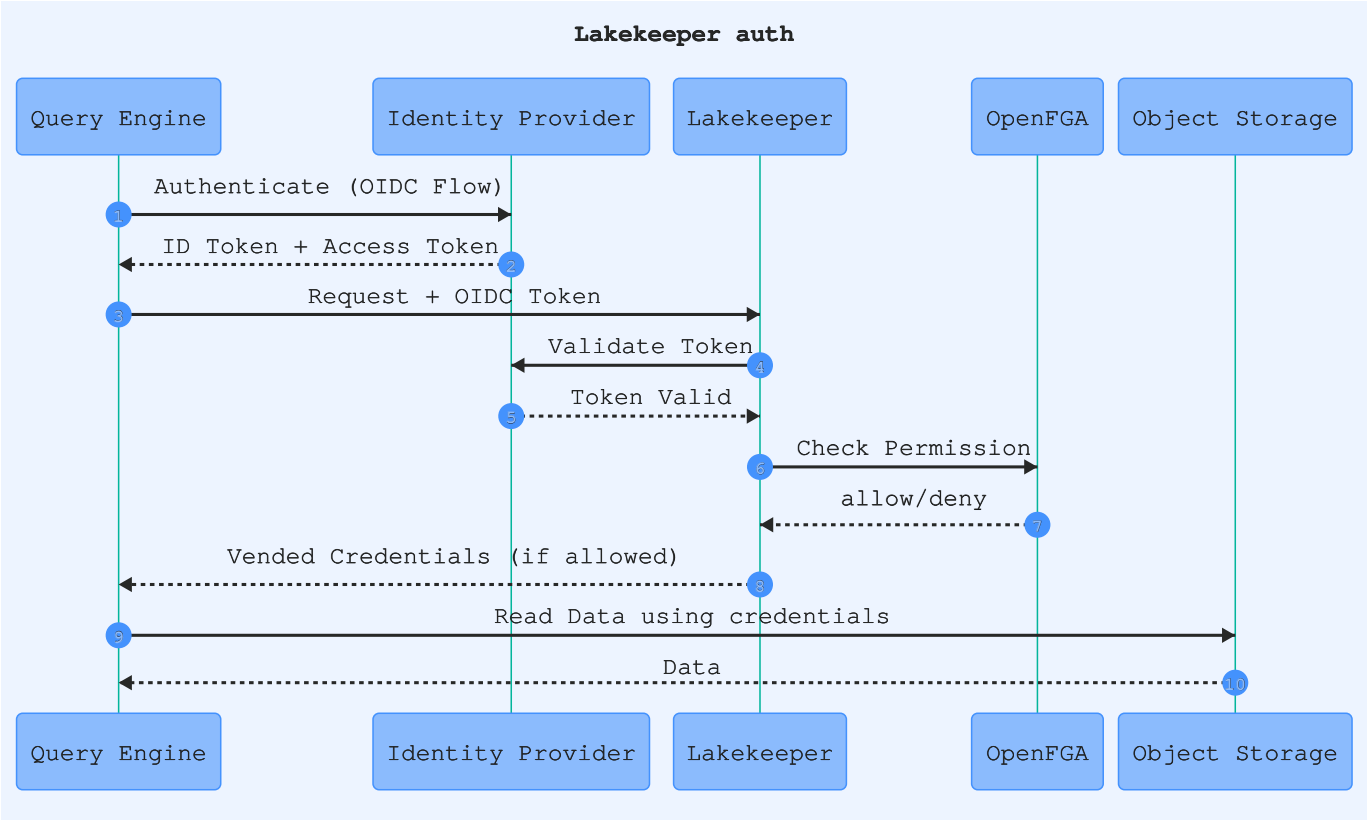

Fig. 4. Authentication & Authorization flow in Lakekeeper

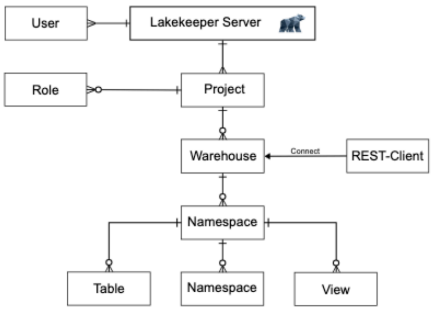

Fig. 5. The hierarchy of Lakekeeper entities. From [3].

of services for managing the platform. As a fully managed cloud platform, it supports storing and querying data through Snowflake's own proprietary SQL execution engine, with security features including RBAC and full end-to-end encryption using industry-standard encryption. Snowflake supports column- and row-level granular access control through the use of **column masking policies** and **row access policies**.

Column masking policy Used to transform the values of a column to an unreadable, "masked" format, to redact protected information. An example could be replacing all values of a column with an asterisk, "*". With this, queries that return information from columns the user does not have access to will have their values masked at query time. [31]

Row access policy Used to govern access to entire rows by filtering out rows the user does not have permission to at query time. Unlike column masks, none of the protected data is returned to the user, even in a masked format, but instead filtered out by the query engine. [32]

On top of being a fully managed "all-in-one" platform, Snowflake also supports querying *external tables* instead of data stored in Snowflake-managed tables, to enable querying of data lakes built on open file formats like Parquet and Iceberg, and allowing for other query engines to operate on the same data. There are many options for how to configure this, but for the scope of this paper, we will focus on the following two options.

Snowflake Horizon Catalog With this, Snowflake creates and manages external Iceberg tables and acts as the catalogue of the data lake, managing all metadata.

Using the Snowflake UI, Iceberg tables can be queried as regular Snowflake tables, with column- or row-level granularity by the use of masking policies and row access policies. External query engines must go through the Horizon catalog, by authenticating using Snowflake access tokens or an External IdP and then querying using the catalogue. However, querying Iceberg tables with column- or row-level security is not supported using external query engines.[33]

Snowflake Open Catalog a managed, cloud-based version of Apache Polaris (Section IV.D), supporting the same interface and access control policies as Polaris. [34]

C. *Databricks (Unity Catalog)*

Unity Catalog is an open-sourced catalogue initially created by Databricks. Unity Catalog is limited in its access control,

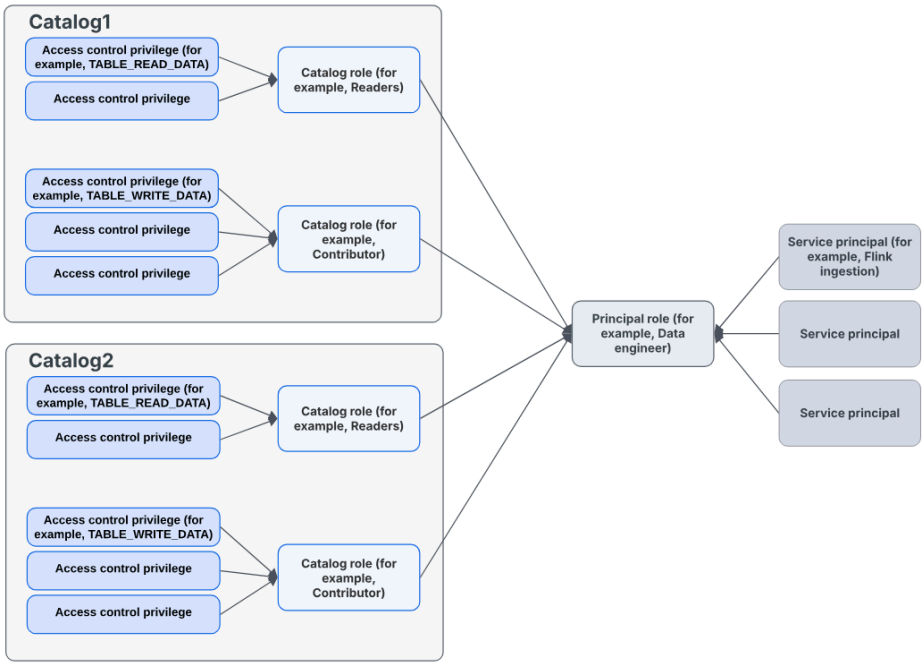

Fig. 6. RBAC model in Apache Polaris, from [44]

and only when using it through Databricks, finer granularity than file-level is enforced.[35]

The flow in Databricks differs from, e.g. Lakekeeper, as by default it does not use vended credentials, but it is supported. Instead, all access is routed through Databricks itself. Meaning that no temporary access is given to the object store from the client's perspective.

This allows Databricks to offer granularity levels at the object, catalogue, schema, table, view, volume, column, and row levels. Row and column level access control are enforced by row filters and column masks [36], [37], which are defined in Section IV.B.

Databricks also supports encryption-at-rest. This is handled by using something similar to envelope encryption, though not specified in detail. [38], [39]

For IdP, Databricks relies on its own internal IdP. Though it does support synchronizing an externally managed IdP with its own internal one. Its internal IdP relies on attribute-based access control (ABAC) and not RBAC.[40], [41]

D. Apache Polaris

Apache Polaris is a catalogue implementation for Apache Iceberg. Figure 6 shows the model of RBAC used, and Apache Iceberg supports REST-compatible query engines, which include platforms like Apache Spark.

To access data, the client/query engine is authenticated through either the default IdP or a configured external IdP, which uses OpenID Connect. The obtained token is then verified by Polaris, and the principal role is resolved based on the token.

It can then check the roles allowed to the user and provides credential vending, which "allow the query engine to run the query without requiring access to your cloud storage for Iceberg tables"[42].

Apache Polaris offers a granularity level starting at the catalogue, namespace level, and continues at the Iceberg Table, View, and Policy level, which "is a structured entity that defines rules governing actions on specified resources under predefined conditions"[43].

Same as with Lakekeeper, Polaris does not offer encryption-at-rest. But as it is also an Iceberg-based catalogue, Parquet files are supported. Parquets encryption can thus be used to ensure encryption-at-rest, but would need a separate cryptographic system in place to work.[42]

E. DuckLake

DuckLake is an "integrated data lake and catalog format"[45]. It uses a relational SQL database as its catalogue database, and an object store such as S3 as a data store, storing data using the Parquet file format.[46] While DuckLake itself does *not* support access control out of the box, it can leverage the access control systems from its catalogue database and object store to provide access control. Using, for example, PostgreSQL and S3, access control can be achieved by using PostgreSQL roles along with AWS IAM roles.[47]

Since access control is not native to DuckLake, the granularity of the permission system depends on the permission system provided by the "building blocks"; i.e., the catalogue database and object store. For example, PostgreSQL can enforce permissions at the schema or table level by using PostgreSQL's roles and `GRANT`s[48]. Then, by using AWS Identity and Access Manager[49], it can be defined that, given some credentials, you are enabled to create, read, update, and delete the given Parquet files.

TABLE I
COMPARISON OF DATA LAKES AND RELATED TECHNOLOGIES AND THEIR FINEST GRANULARITY FGAC SUPPORT

Format	Type	Encryption-at-Rest	IdP Support	Finest granularity FGAC
Parquet	File format	Yes	Not applicable	Row level
Lakekeeper	Catalogue	Yes (only through Parquet)	External	Table/view
Apache Polaris	Catalogue	No	Internal and external (if OIDC)	View
Databricks	Data lake	Yes	Internal and external sync	Column/row
Snowflake Cloud	Data lake	Yes	Internal	Column/row
DuckLake	Data lake and catalogue format	Yes	External	Table

By doing the above, DuckLake can rely on object-store encryption (e.g., S3 SSE-KMS) and/or Parquet file-level encryption.

F. Direct Comparisons of data lake technologies

The different technologies have various advantages and disadvantages. Table I shows the overall differences as well as the finest granularity supported by FGAC, supported by the format.

As Parquet is a file format, it cannot enforce AC to any extent, but it does support encryption, which, if coupled with other systems, can enforce AC.

It is important to note that the encryption-at-rest for the solutions is only supported if the whole system is used; thus, if another data lake system accesses the object store, they will not be able to read decrypted data. This is a big issue, as whoever wants to access the data is forced to use the data lake system.

From Section IV.F, we can see that in terms of access control, the solutions generally fall into two categories:

a) *Vended credentials from the catalogue:* Access control is enforced through the catalogue's vending of scoped credentials pertaining to the permissions of the query engine.

b) *Fully managed cloud platform:* Access control is managed by the query engine and the catalogue since they are both under the fully managed platform. This means the query engine is trusted to filter out rows that are not permitted and mask columns that are protected. This is obviously not possible on external query engines since they cannot be trusted to perform the necessary filters and masks.

Solutions like Lakekeeper and Polaris create vended credentials to S3, but since the Object Store can store Parquet files, which contain many columns, and S3 does not know anything about the contents of the files, Lakekeeper will not be able to govern access to these columns. On the other hand, all-in-one solutions like Databricks or Snowflake only support the use of their own query engine and proprietary file formats when using their full cloud platform, or when using external tables, or provide a solution that is reminiscent of the "vended credentials" solution, like Lakekeeper or Polaris.

This leaves a gap in the case where column-level access control is wanted, on files in a data lake, but open file formats like Iceberg or Parquet are used. The solution for enabling this has to somehow enforce AC post-retrieval of the files, so that regardless of the query engine trust level, it is forced to comply with the granularity. This would allow for anyone trusted to access the file to be able to access the file, even though some columns might not be entrusted to the user.

V. FINE-GRAINED ROLE-BASED ACCESS CONTROL BY ENCRYPTION

Fine-grained role-based access control (FGRBAC) by encryption is our proposal for a solution that ensures a more varied choice of query engines can be used by encrypting the data-at-rest, and all authenticated access to the object store. This can be done by leveraging the supported encryption with Parquet. Resulting in (given a separate system which can help decrypt the data) users being able to access columns only meant to be read by them, whilst the whole system does not have to trust the query engine implementations itself.

In the next subsections, we will describe the logic behind reading and writing using our proposed solution.

a) *Encrypting (create/update):*

On insert/update, cryptographic keys are generated for each column, where RBAC is required, and the data is encrypted, together with their respective column metadata in the footer. The reasoning for encrypting the column metadata was discussed in Section II.H.a, but it is mainly due to being able to contain sensitive information. Then the cryptographic keys will get an ID, the ID will be stored in the footer metadata for its column, and the key ID and key will be stored as a pair in the KMS.

Example: Suppose Alice wants to create/update a new table with the following columns: `names`, `ages`, and `CPR`. Alice might want to ensure that only users with the role `manager` can access `names` and `CPR`, whilst ages can be accessed by everyone who has access to the S3 object store. Then Alice will create two cryptographic keys, one for `names` (`name_key`) and one for `CPR` (`cpr_key`). Alice can now encrypt the columns and their respective footer column metadata, with their respective key, and leave the `ages` column decrypted. In the Parquet footer metadata, Alice will add the IDs of `name_key` and `cpr_key`. The key ID and the key itself are then added as a pair to the KMS and linked to the role `manager`.

b) *Decrypting (read):*

On read, the Parquet file, which contains the wanted data, might be partially encrypted. The cryptographic keys will need to be retrieved from the KMS to then decrypt the data

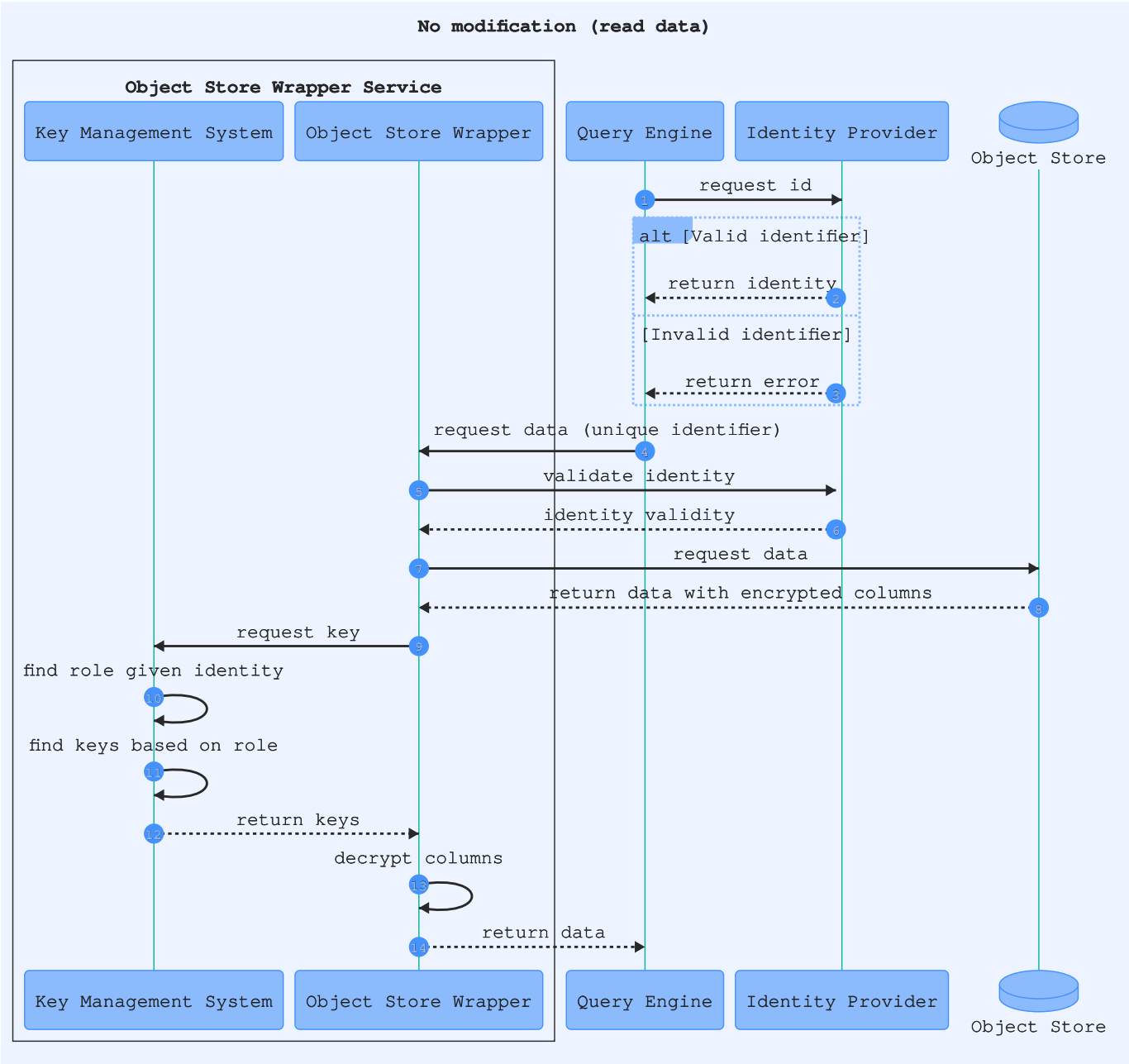

Fig. 7. Sequence Diagram showing how an unmodified data lake query engine gets data from the object store

and return the decrypted file. Though it might still be partially encrypted.

Example: Suppose Bob wants to read from a table with a role which is not `manager`, Bob can retrieve the Parquet file (which now has two encrypted columns). Bob then ask the KMS for the key IDs of the columns `name_key` and `cpr_key`, but as Bob's current role is not linked to those keys Bob does not get them. Thus, Bob can only read the `ages` column. If, however, Bob had the `manager` role, then the KMS would return the keys, and Bob would be able to decrypt the columns, allowing Bob to see all the data.

Though there is now an issue. How do we ensure that current data lake solutions do not need to make changes to support encryption and decryption, as most already handle authorization and AC in their own ways?

Our proposal includes an Object Store Wrapper Service (OSWS). This will essentially mock S3 and ensure that current data lake solutions just have to change the connection string related to S3 to the OSWS one, meaning that it will use the S3 API, but not be an actual S3 object store internally.

To read from the object store, you are then provided with two options.

Option 1: Go through the provided OSWS.
Option 2: Connect directly to the KMS, which is served through the OSWS, and add a decryption step to your query engine.

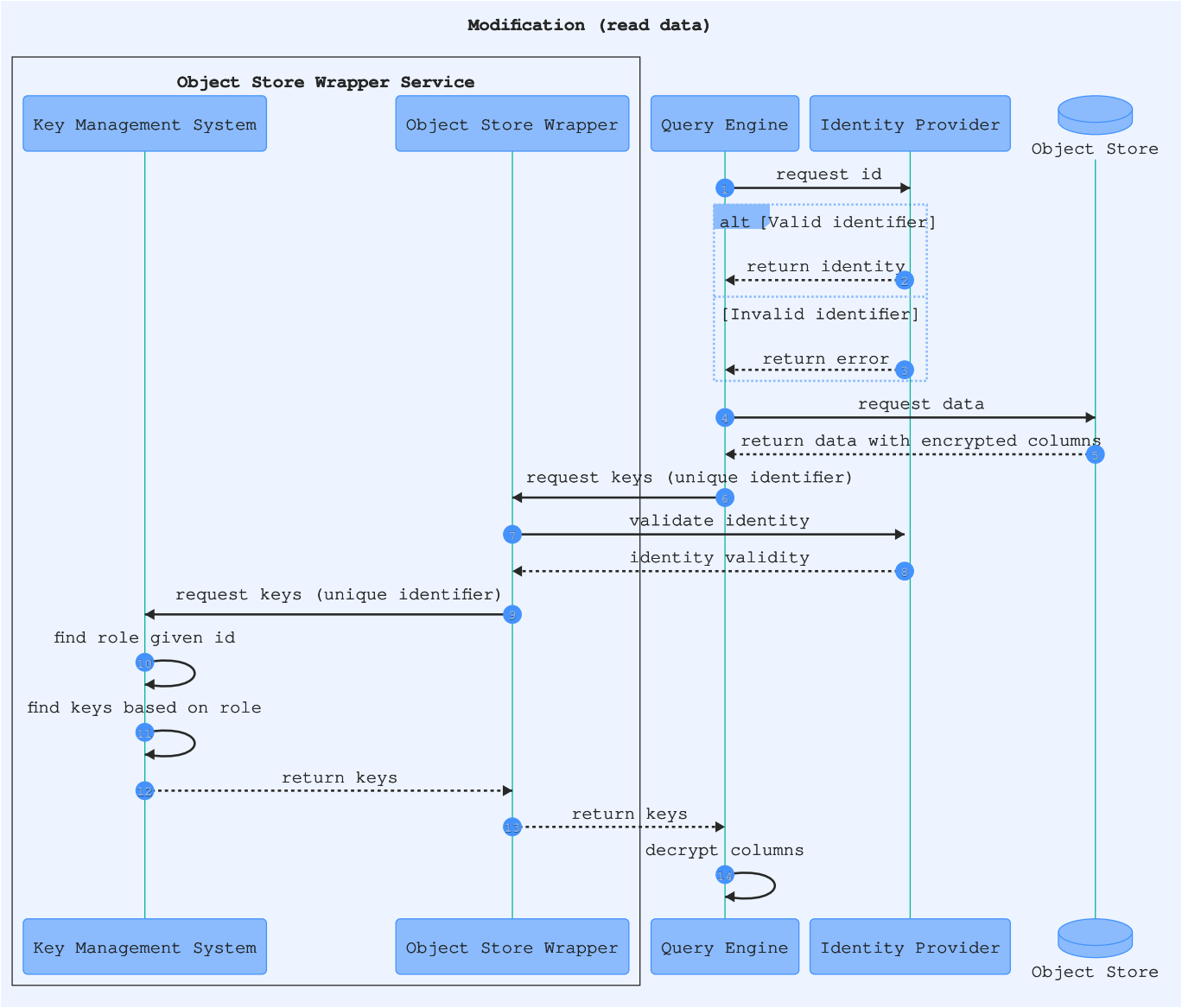

Fig. 8. Sequence Diagram showing how a modified data lake query engine gets data from the object store

The idea of the OSWS **Option 1** can be seen in Figure 7 and its steps. The Query Engine will fetch an authentication token from the IdP, steps 1 to 3. It will then make its request to the OSWS, step 4, where the OSWS can then validate the token against the IdP, step 5 and 6. OSWS will then request the data from the object store, and on return, it can figure out which cryptographic keys are needed to decrypt the columns, steps 7 and 8. It then requests the keys, together with the provided identity, from the KMS, which only returns the keys available for the specific user, and allows to decrypt the columns the user has access to, steps 9 to 12. The OSWS is now able to decrypt the columns, provided the user's access, and can return the file where the columns the user has access to have been decrypted, steps 13 and 14. In short, given the user's authentication token, it can request the KMS for the keys needed to decrypt the entire file, and the KMS will only return the keys for which the specific user has authorization. Thus, regardless of the data lake's own authorization implementation, the OSWS will ensure that all unauthorized data remains encrypted.

A potential issue with this approach is the overhead introduced by cryptographic operations on potentially large Parquet files. This will have to be benchmarked thoroughly in an implementation.

Another issue is that it might only be a single column in the given Parquet file that the client is interested in, thus the OSWS will end up doing more work than necessary, and this is not something which can be addressed due to the limitations in the S3 API.

We propose **Option 2** to address the overhead, which needs more logical steps in the query engine by the user, which can be seen by the steps on Figure 8. The query engine will authenticate itself against the IdP and then query S3 directly, and it will return the possibly partially encrypted Parquet files, steps 1 to 5. Given the metadata of the Parquet files' footers and which columns it needs to read, it can then request the

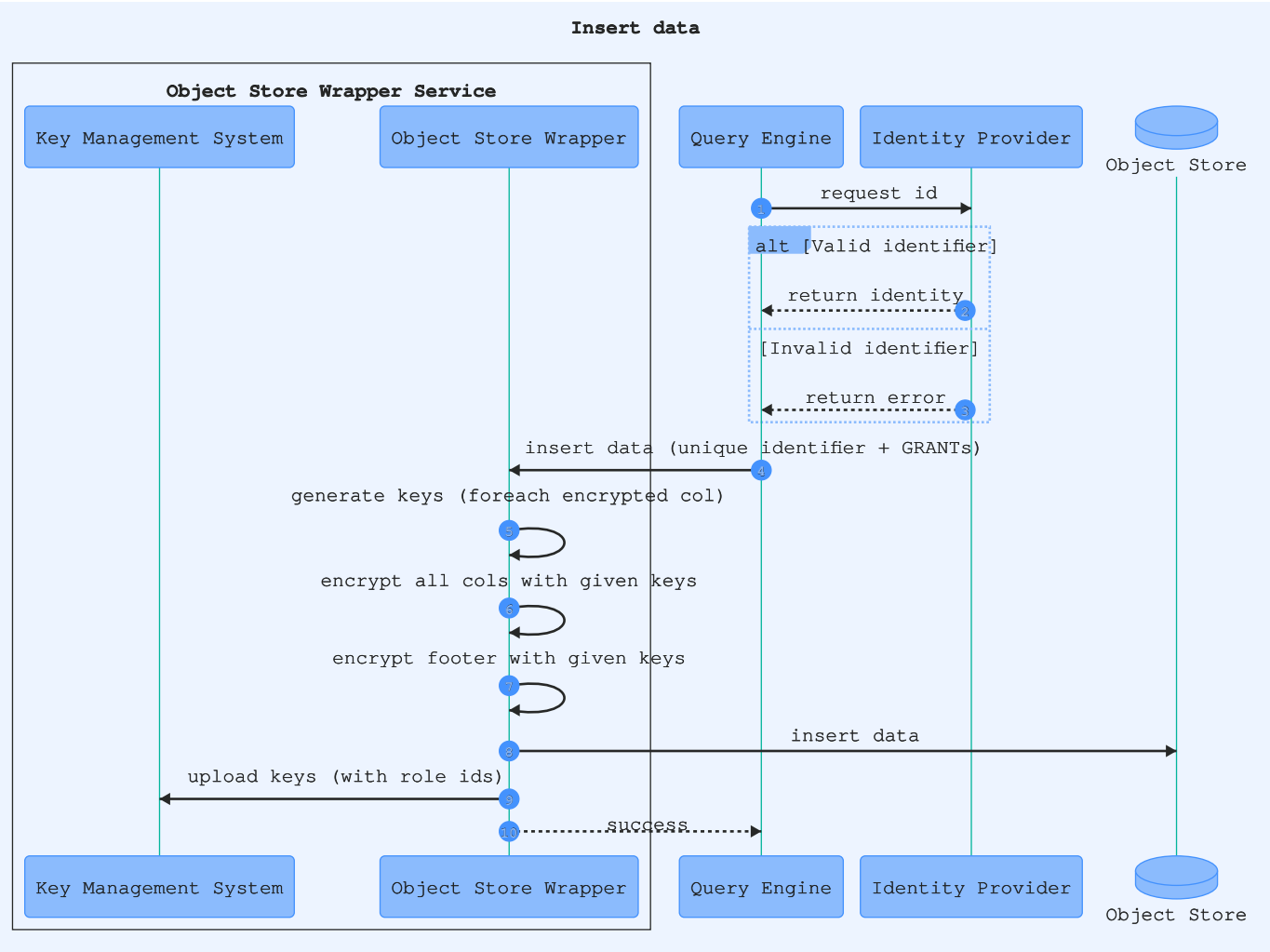

Fig. 9. Sequence Diagram showing how an unmodified data lake query engine inserts data into the object store

OSWS extra endpoint for the KMS, step 6. The endpoint then first validates the identity against the IdP and then requests the KMS for the specified keys, which, given the user has the correct roles for the given keys, will be returned to the OSWS, steps 7 to 12. Lastly, the OSWS returns the keys to the user and enables them to decrypt the columns, steps 13 and 14.

Both options 1 and 2 then ensure RBAC regardless of which option you choose, and which query engine you might have. Regardless of which data lake system you choose, or if you create your own implementation, you will be able to retrieve data from the object store. The data will, if needed, be encrypted at rest, and the encryption will be part of ensuring RBAC in the system.

c) *Issues:*

There are a few issues with this approach. These include the safety of the encryption, as well as how we manage `Create` and `Update` to the object store. `Delete` is managed as usual; if you have the given permissions, you can make the call to delete through the OSWS, though the KMS will not be needed for this, but only the authorizations in S3 itself.

For the encryption part, we refer to the Parquet documentation[17], which specifies that the Parquet files support AES. This might be a potential security vulnerability, since nothing stops a malicious actor with authorization to the object store, but not the columns, from downloading the whole object store and then trying to decrypt it. Though this is slightly out of scope, and currently, AES[18] is considered safe and is one of the industry standards for encryption, and it is always possible to increase its key size.

The other issue regarding `Create` and `Update` is how to handle which cryptographic keys to use. The easier one to handle is `Update`. When *updating* a table, the OSWS create new cryptographic keys that have the same roles as the previous cryptographic keys used to encrypt the columns. It ensures that if some cryptographic keys are leaked, it is limited to only a specific version of a table, while also ensuring that the same roles can still access the data.

Calling `Create` through the OSWS will, by default, not encrypt the columns, though given that the admin wants to

[17] https://parquet.apache.org/docs/file-format/data-pages/encryption/#44-additional-authenticated-data

[18] AES being the abbreviation for “Advanced Encryption Standard”

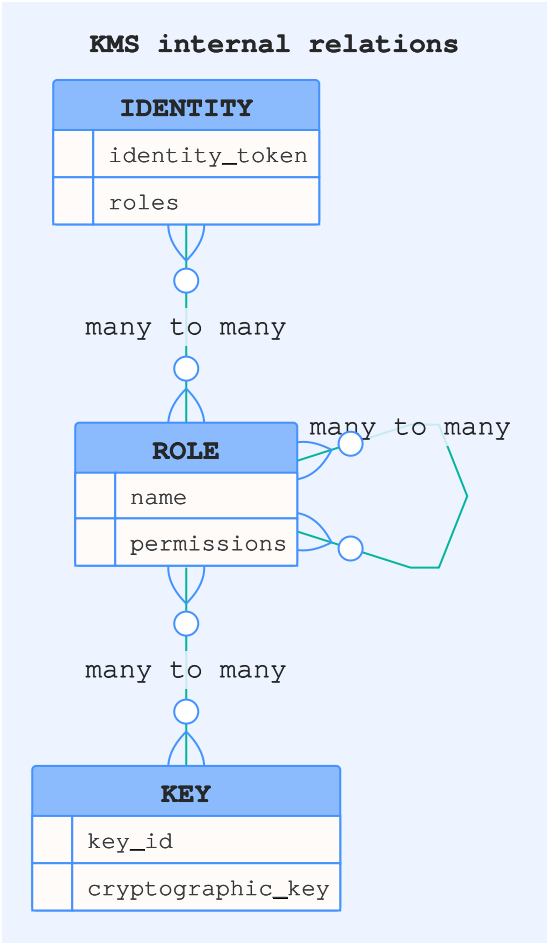

Fig. 10. How the relations internally are mapped in the KMS

ensure all files are encrypted, a default role can be created and all columns without `GRANT`s can be encrypted with said role. If the user uses the `GRANT` keyword for columns, the OSWS will encrypt the columns with newly generated cryptographic keys, seen on steps 4 to 7 in Figure 9. The Parquet files will be added to S3, and the keys will then be added to the KMS and put under the role which was specified in the `GRANT`, thus ensuring that they can be read by others who have the given role, as seen on steps 8 and 9.

By only allowing `Create`, `Update`, and `Delete` through the OSWS, we can ensure that the object store is used only as an immutable storage. When updating a Parquet file, deleting it or similar, the OSWS will make new versions. It will ensure that authorized malicious actor can break or delete data, and it will always be possible to roll back.

As we use RBAC, the KMS needs to map several IDs, which map to roles, roles can map to other roles, which then map to a large number of tuples of cryptographic key IDs and the cryptographic keys themselves. Figure 10 displays are a more precise version of the internal relations, and they are not hierarchical. It is also important to note that the KMS will serve as the authorization manager.

d) *Functional Requirements:*

1) Authentication and ID verification
The OSWS validates the user, and it supports common standards, e.g. OpenID Connect (OIDC).
2) KMS
The KMS must serve as authorization manager and map a user's identity to their roles, which map to the available keys.
3) Enforcement of Access Control
Access control is ensured by that the KMS only returns the cryptographic keys to the columns that the user is allowed to read.
4) Data Decryption
Option 1: If using an unmodified query engine, the OSWS must handle the decryption of the requested file before returning the file to the unmodified query engine.
5) Lazy Data Decryption
Option 2: The system must support that the query engine fetches directly from the object store, to later fetch the cryptographic keys through the OSWS to decrypt the available columns.
6) Encrypted metadata
The system must, on encryption of columns, encrypt the related statistical metadata, as it might contain sensitive information.
7) Key Generation
For updates, OSWS must create new cryptographic keys for the columns and link the keys in the KMS to the same previous roles.
8) Conditional Encryption on `Create`
The OSWS should only encrypt columns, and their respective statistical metadata, given the user has specified a role in the `GRANT`for the column.
9) Write authorization and cryptographic key storage
If a `GRANT` is specified during the `Create` operation, the OSWS must encrypt the columns with a generated cryptographic key. It then has to be stored in the KMS under the role specified in the `GRANT`
10) Parquet Metadata
Given a column is encrypted, its column-specific metadata in the Parquet footer must contain the cryptographic key ID, so it is known which cryptographic key is needed to decrypt the column.
11) Object Store Mocking
The OSWS has to mock S3, so existing data lake solutions only have to change their connection string to the OSWS one.
12) Immutability
The OSWS has to ensure the object store is used as it is immutable. Thus allowing for roll-backs in case of wrong modifications to the stored data.

e) *Non-functional Requirements:*

1) FGAC
OSWS has to enable FGAC on a column-level granularity level, by using RBAC and leveraging the Parquet encryption.
2) Security (encryption-at-rest)
OSWS has to enable encryption-at-rest for the data

stored in the object store, regardless of which data lake solution is used.

3) Security (key policy management)
The KMS in the OSWS has to enforce key policies.
4) Compatibility
The OSWS should allow for the solution to be catalogue agnostic, meaning that only the object store layer needs to be changed, thus allowing current Lakekeeper and Databricks to use the solution.
5) Performance
The solution should prioritize performance by making others choose to modify their query engine to use *Option 2*, specified in Section V.-.b, to allow for lazy decryption of the columns, thus reducing the overhead for the user.
6) Security (Vulnerability)
The encryption scheme should use a strong symmetric encryption, like AES, supported by Parquet.
7) Scalability
The KMS has to be able to handle the potentially large complexity of many interlinked users, roles, and tuples of cryptographic key IDs and their cryptographic keys.

f) *OSWS conclusion:*

It is important to reiterate that OSWS does not solve all the issues that exist with data lake systems. Its main advantage is to enforce column-level AC, even when query engines which do not implement RBAC themselves connect to it, while still allowing existing data lakes to use the system.

The proposed OSWS does not address AC on row-/cell-level. This was a result of wanting to use a pre-existing file format, to ensure as easy an adoption as possible. In the future, this could be explored to see if the Parquet file format could be extended or if another proposed file format could support finer granularity.

VI. Discussion

This leads to what are the general advantages, disadvantages, and points which have to be explored further for the FGRBAC by Encryption Section V to work properly. The following subsections will address some of these points.

A. No lakehouse needed

Many of the discussed data lake solutions use a "lakehouse" structure to govern access to the data itself. The catalogue acts as the security layer, controlling whether the query engines are allowed to do what they do, and giving out access tokens depending on this. This is the "Vended credentials from the catalogue" category as described in Section IV.F. This is the case for Lakekeeper (Section IV.A) as well as for Databricks with Unity Catalog (Section IV.C). This has the downside of *requiring* query engines to go through the catalogue to get the data they want, which doesn't allow for "schema-on-read" query operations on the raw data. This is by design – since the object stores might contain different tables with many columns, and the catalogues can only give access to what files the query engine can read, the catalogue has no way to control what columns the query engine can read. Databricks solves this by having the query engine filter out what the reader is not allowed to access, using row filters and column masks [50]. However, this is only possible because Databricks is in control of the query engine. For a reader who wants to query the data lake "directly" with their own query engine, this would not be possible. With the proposed solution of (Section V), controlling access on the column level would be possible even for "schema-on-read" operations, since the access is governed at the object store, and not the catalogue. This means a data lake using the proposal does not need a specific, secure catalogue to ensure access control.

B. Two birds, one stone

Like with the solution proposed by Shang et.al. [21], the solution inherently solves both access control and encryption-at-rest, meaning that there is no need for authorization systems like OpenFGA as Lakekeeper (Section IV.A) uses on top of the encryption.

Next, we discuss some potential shortcomings and downsides of the solution.

C. Performance

Since the solution needs to perform cryptographic operations in the OSWS, performance might become an issue. This can partly be helped by the query engines themselves fetching the decryption keys together with accessing the object store directly – thus allowing for lazily decrypting the data. The proposed solution would need to be benchmarked heavily to decide if the overhead of cryptographic operations is significant.

D. Security

The status quo for data lakes is typically either to deny access to the entire file due to authorization on the finer-grained data or physically remove the columns or rows that a user is not permitted to see.

Our proposed system would change this, and given that you have access to some of the files, you would see the file. The difference is that some decryption might be needed to get usable data from the columns. It might be discussed if this is a safe solution, as the user might get files with encrypted columns (which they are not authorized to read) and then try to hack their way around it.

Furthermore, the OSWS would be the single point of trust, which has to be analyzed to see if this solution is safe.

The proposal further relies on the integrity of the OSWS and the KMS. Compromising one component would allow unauthorized decryption of protected columns. This will have to be analyzed in depth to ensure no attack vectors, but should be addressed in future work.

E. Shortcomings

The proposed solution only has column-level granularity support. If cell-level granularity is wanted, two main options are available. Either the Membrane proposal Section III.B could be implemented, though it would need a lot of custom implementation, and would make it difficult for low-level

query engines to create a decryption step, due to the extent of the theory behind the proposal. The other option is that the Parquet file specification gets extended to support cell-level encryption. This would be simpler for low-level query engines, as it might work the same way the current OSWS proposal works, by saving the cryptographic key IDs in the Parquet files' metadata.

VII. Future Work

For future work, we propose to create an MVP for the OSWS system, described in Section V, as well as describe in detail the standard used to handle the RBAC-aware column-level granularity decryption.

Afterwards we would integrate the following two systems: **System 1**, being an unmodified query engine, thus it has to use *Option 1* and fetch its requested data through the OSWS, and **System 2** being a system using *Option 2* and thus handles decryption locally and communicates only with the KMS in the OSWS and directly with the object store. *System 1* should add the OSWS as the intermediate between a previously set-up data lake, between the query engine layer and the object store layer. *System 2* should, instead of adding the OSWS as the intermediate, extend its query engine with the extra decryption step.

It is important that both *System 1* and *System 2* are based on the same data lake, as this will allow for direct benchmarking between the unmodified data lake and the modified one, and see what trade-offs each system offers.

While this is being developed, the documentation will be important. This allows for future development by third parties and ensures that the standard can be adopted. The more that implement the system, the more likely it will be that existing and future data lakes will directly implement the use of *Option 2*, thus ensuring the OSWS has to handle as little work as possible.

VIII. Conclusion

We set out to find a way to unify access control in data lakes, as it is currently enforced in different ways depending on the solution chosen and how it handles its granularity. Thus, the main motivation behind the project was to ensure that the concept of data lakes could be used by a wider array of systems and ensure fine-grained access control, regardless of the implementation on the user's system.

We reviewed existing and proposed solutions as well as the theory used within the space of data lakes. The systems were compared to ensure a varied understanding of the trade-offs. Both papers discussed in Section III also showed how encryption-at-rest might be used to ensure fine-grained access control, meaning that the idea of fine-grained access control by encryption is not a completely new and novel idea.

We propose a new standard and system, which is role-based access control aware and has fine-grained access control to a column-level granularity. It should have interoperability with existing systems, but can be more efficient with a bit of custom implementation. We call this "Fine-grained role-based access control by encryption"

Further work will include implementing this system in a well-documented and open-sourced manner to ensure that it can easily be adopted. Furthermore, the system should be benchmarked both in terms of speed and security to determine its usability. The possibility of row-level and cell-level access control should also be explored further, as it would result in even more use cases for data lake systems.

References

[1] AWS, "Governance and access control for Apache Iceberg on AWS," [Online]. Available: https://docs.aws.amazon.com/prescriptive-guidance/latest/apache-iceberg-on-aws/governance.html

[2] Microsoft, "Access control model in Azure Data Lake Storage," 2024, [Online]. Available: https://learn.microsoft.com/en-us/azure/storage/blobs/data-lake-storage-access-control-model

[3] the Lakekeeper Team, "Documentation/Concepts." 2025.

[4] A. S. Foundation, "Documentation," [Online]. Available: https://iceberg.apache.org/docs/1.5.0/

[5] [Online]. Available: https://www.ibm.com/think/topics/data-lake

[6] D. F. Ferraiolo and R. Kuhn, "Role-Based Access Controls," in *Proceedings of the 15th National Computer Security Conference (NCSC)*, Baltimore, MD, USA, Oct. 1992, pp. 554–563.

[7] D. F. Ferraiolo, R. Sandhu, S. Gavrila, D. R. Kuhn, and R. Chandramouli, "Proposed NIST standard for role-based access control," *ACM Trans. Inf. Syst. Secur.*, vol. 4, no. 3, pp. 224–274, Aug. 2001, doi: 10.1145/501978.501980.

[8] M. Azure, "Azure Data Encryption At Rest; What is Encryption at Rest?."

[9] [Online]. Available: https://docs.aws.amazon.com/AmazonS3/latest/userguide/Welcome.html

[10] A. S. Foundation, "Apache Spark."

[11] M. A. Zaharia, A. Ghodsi, R. Xin, and M. Armbrust, "Lakehouse: A New Generation of Open Platforms that Unify Data Warehousing and Advanced Analytics," in *Conference on Innovative Data Systems Research*, 2021. [Online]. Available: https://api.semanticscholar.org/CorpusID:229576171

[12] [Online]. Available: https://www.cloudflare.com/learning/access-management/what-is-an-identity-provider/

[13] [Online]. Available: https://www.ibm.com/docs/en/db2/12.1.x?topic=security-row-column-access-control-rcac

[14] C. C. Editor, "Key Management System - Glossary | CSRC." [Online]. Available: https://csrc.nist.gov/glossary/term/key_management_system

[15] [Online]. Available: https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Overview.Encryption.Keys.html

[16] [Online]. Available: https://www.ibm.com/think/topics/key-management

[17] A. S. Foundation, "Apache Iceberg™ Documentation — Latest."

[18] Apache, "Parquet File Format."

[19] A. S. Foundation, "Apache Thrift."

[20] [Online]. Available: https://parquet.apache.org/docs/file-format/data-pages/encryption/

[21] X. Shang *et al.*, "One Stone, Three Birds: Finer-Grained Encryption with Apache Parquet @ Large Scale," in *2022 IEEE International Conference on Big Data (Big Data)*, Osaka, Japan: IEEE, Dec. 2022, pp. 5802–5811. doi: 10.1109/BigData55660.2022.10020987.

[22] S. Kumar, S. Yagati, C. Power, D. E. Culler, and R. A. Popa, "Membrane: A Cryptographic Access Control System for Data Lakes," *ArXiv*, 2025, [Online]. Available: https://api.semanticscholar.org/CorpusID:281243750

[23] [Online]. Available: https://auth0.com/intro-to-iam/what-is-openid-connect-oidc

[24] C. N. C. Foundation, "Introduction to OpenFGA." 2025.

[25] R. Pang *et al.*, "Zanzibar: Google's Consistent, Global Authorization System."

[26] barclayn, "What is Microsoft Entra? - Microsoft Entra." [Online]. Available: https://learn.microsoft.com/en-us/entra/fundamentals/what-is-entra

[27] the Lakekeeper Team, "Authorization," [Online]. Available: https://docs.lakekeeper.io/docs/nightly/authorization/
[28] the Lakekeeper Team, "Documentation/Storage." 2025.
[29] Amazon, "Amazon Security Token Service API Reference."
[30] B. Dageville *et al.*, "The Snowflake Elastic Data Warehouse," in *Proceedings of the 2016 International Conference on Management of Data*, San Francisco California USA: ACM, June 2016, pp. 215–226. doi: 10.1145/2882903.2903741.
[31] Snowflake, "Understanding Column-level Security."
[32] Snowflake, "Understanding row access policies."
[33] Snowflake, "Query Apache Iceberg™ tables with an external engine through Snowflake Horizon Catalog."
[34] Snowflake, "Snowflake Open Catalog overview."
[35] [Online]. Available: https://www.databricks.com/company/newsroom/press-releases/databricks-open-sources-unity-catalog-creating-industrys-only-open
[36] [Online]. Available: https://docs.databricks.com/aws/en/data-governance/unity-catalog/manage-privileges/
[37] [Online]. Available: https://docs.databricks.com/aws/en/data-governance/unity-catalog/filters-and-masks/
[38] [Online]. Available: https://docs.databricks.com/aws/en/security/keys/kms-s3
[39] [Online]. Available: https://www.databricks.com/blog/2021/05/24/security-best-practices-for-aws-on-databricks.html
[40] [Online]. Available: https://docs.databricks.com/aws/en/admin/users-groups/scim/
[41] [Online]. Available: https://docs.databricks.com/aws/en/data-governance/unity-catalog/abac/
[42] [Online]. Available: https://polaris.apache.org/releases/1.2.0/
[43] [Online]. Available: https://polaris.apache.org/releases/1.2.0/policy/
[44] [Online]. Available: https://polaris.apache.org/releases/1.2.0/managing-security/access-control/
[45] G. User, "DuckLake is an integrated data lake and catalog format." [Online]. Available: https://ducklake.select/
[46] G. User, "Introduction." [Online]. Available: https://ducklake.select/docs/stable/specification/introduction.html
[47] G. User, "Access Control." [Online]. Available: https://ducklake.select/docs/stable/duckdb/guides/access_control.html
[48] [Online]. Available: https://www.postgresql.org/docs/18/sql-grant.html
[49] [Online]. Available: https://docs.aws.amazon.com/IAM/latest/UserGuide/introduction.html
[50] Databricks, "Unity Catalog attribute-based access control (ABAC)."